Лекция 9.
Транспортный уровень: TCP, UDP и порты

Транспортный уровень в сетевой модели
TCP и UDP
Порты, socket и диагностика сервисов

Цель лекции

- Понять назначение транспортного уровня.
- Уметь объяснять различия TCP и UDP.
- Понимать назначение портов и socket.
- Различать проверку IP-связности и доступности приложения.
- Уметь выбирать базовые команды диагностики транспортного уровня.

Список учебных вопросов

1. Назначение транспортного уровня.
2. TCP: Transmission Control Protocol, протокол с установлением соединения.
3. UDP: User Datagram Protocol, протокол без установления соединения.
4. TCP handshake на базовом уровне.
5. Метод скользящего окна на уровне назначения.
6. Socket: сочетание IP-адреса, протокола и порта.
7. Well-known ports популярных сервисов.
8. Отличие проверки IP-связности от проверки доступности приложения.
9. Диагностика транспортного уровня.

1. Назначение транспортного уровня

- Транспортный уровень передает данные между приложениями на разных узлах
- Узел - компьютер, сервер, маршрутизатор или другое устройство в сети
- IP-адрес помогает доставить пакет до нужного узла
- Порт помогает доставить данные до нужной программы внутри узла
- Без транспортного уровня сеть не различала бы веб-сервер, DNS, SSH и другие службы на одном хосте

Место транспортного уровня в модели

В модели OSI транспортный уровень находится выше сетевого уровня.

В стеке TCP/IP транспортный уровень работает поверх IP.

IP отвечает за доставку между узлами.

TCP или UDP отвечают за обмен между программами.

Приложения используют транспортный уровень через сетевые сокеты.

ТРАНСПОРТНЫЙ УРОВЕНЬ МЕЖДУ IP И ПРИЛОЖЕНИЯМИ

Транспортный уровень обеспечивает сквозную связь между приложениями, работая поверх сети Интернет (IP).

МОДЕЛЬ OSI

7. **Прикладной** (Процессы, приложения)
6. **Представительный** (Форматы данных, шифрование)
5. **Сеансовый** (Управление сеансами)
4. **Транспортный** (TCP, UDP)
3. **Сетевой** (IP, маршрутизация)
2. **Канальный** (Доступ к среде)
1. **Физический** (Передача битов)

ПРИКЛАДНОЙ УРОВЕНЬ (ПРОЦЕССЫ И ПРИЛОЖЕНИЯ)

- Веб-браузер (HTTP/HTTPS)
- Электронная почта (SMTP/IMAP/POP3)
- Потоковое видео (Netflix, YouTube)
- VoIP / Звонки (SIP, RTP)
- Передача файлов (FTP, SFTP)
- Удалённый доступ (SSH, Telnet)
- ...

Данные приложений

ТРАНСПОРТНЫЙ УРОВЕНЬ

TCP (Transmission Control Protocol)

Ориентирован на соединение

- Надёжная, ориентированная на соединение передача
- Гарантия доставки и упорядочивания данных
- Управление потоком (flow control)
- Контроль ошибок и повторная передача
- Механизм «скользящего окна»

UDP (User Datagram Protocol)

Без установки соединения

- Ненадёжная, без установления соединения
- Быстрая передача с минимальными накладными расходами
- Нет гарантий доставки и упорядочивания
- Подходит для потокового вещания, DNS, игр и VoIP

Сегменты (TCP) / Датаграммы (UDP)

СЕТЕВОЙ УРОВЕНЬ (IP)

- Логическая адресация (IPv4 / IPv6)
- Маршрутизация пакетов между сетями
- Доставка пакетов от хоста к хосту

Передача через Интернет

ЗАЧЕМ НУЖЕН ТРАНСПОРТНЫЙ УРОВЕНЬ?

- Сквозное взаимодействие между приложениями на разных хостах
- Выбор нужного приложения на принимающей стороне (порты)
- Надёжность и управление потоком (для TCP)
- Оптимизация скорости и минимизация накладных расходов (для UDP)

ПОРТЫ – АДРЕСА ПРИЛОЖЕНИЙ

Транспортный уровень использует номера портов для доставки данных конкретному приложению.

80	HTTP	53	DNS
443	HTTPS	22	SSH
25	SMTP	5060	SIP

...

СРАВНЕНИЕ TCP И UDP

Характеристика	TCP	UDP
Тип соединения	Ориентирован на соединение	Без соединения
Надёжность	Надёжный	Ненадёжный
Порядок данных	Гарантируется	Не гарантируется
Контроль потока	Есть	Нет
Скорость	Ниже (больше накладные расходы)	Выше (меньше накладные расходы)
Примеры	HTTP/HTTPS, FTP, Email, SSH	DNS, VoIP, Видео, Онлайн-игры

ИНКАПСУЛЯЦИЯ ДАННЫХ (ОТПРАВКА)

1. Данные приложения
2. Добавляется TCP/UDP заголовок — [TCP/UDP Заголовок | Данные]
3. Добавляется IP заголовок — [IP Заголовок | TCP/UDP Заголовок | Данные]
4. Добавляется заголовок канального уровня — [L2 Заголовок | IP Заголовок | TCP/UDP Заголовок | Данные | L2 Трейлер]
5. Передача по физической среде — 1 0 1 0 1 0 1 0 1 0

КЛЮЧЕВАЯ ИДЕЯ

Транспортный уровень – это «мост» между приложениями и сетью: он принимает данные от приложений, разбивает их (при необходимости), добавляет служебную информацию и передаёт через сеть, а на принимающей стороне собирает и доставляет нужному приложению.

Что решает транспортный уровень

- Мультиплексирование - одновременная работа многих сетевых приложений на одном узле
- Демультиплексирование - передача полученных данных конкретному приложению
- Сегментация - разбиение большого потока данных на части
- Контроль ошибок - проверка повреждения данных при передаче
- Управление надежностью - выполняется TCP, но не выполняется UDP

Данные на транспортном уровне

- TCP передает данные в виде сегментов
- Сегмент - часть TCP-потока с заголовком TCP и полезной нагрузкой
- UDP передает данные в виде датаграмм
- Датаграмма - самостоятельный блок данных UDP с заголовком и полезной нагрузкой
- Полезная нагрузка - данные приложения, например HTTP-запрос или DNS-запрос

Почему это важно администратору

- Сервис может быть установлен, но не слушать нужный порт
- IP-связность может работать, но приложение может быть недоступно
- Межсетевой экран может пропускать ICMP, но блокировать TCP или UDP
- Ошибка может быть не в адресации, а в службе, порте или правилах доступа
- Диагностика начинается с понимания, на каком уровне возник сбой

2. TCP: Transmission Control Protocol

TCP - Transmission Control Protocol, протокол управления передачей.

TCP обеспечивает надежную передачу данных между приложениями.

Протокол с установлением соединения сначала создает логическую связь.

Данные TCP передаются в правильном порядке.

Потерянные сегменты TCP могут быть переданы повторно.

Когда используют TCP

- HTTP и HTTPS для веб-сайтов
- SSH для удаленного администрирования Linux
- RDP для удаленного рабочего стола Windows
- SMTP, IMAP и POP3 для почтовых служб
- SMB для доступа к общим папкам Windows

Основные свойства TCP

- Надежность - получатель подтверждает прием данных
- Упорядоченность - приложение получает данные в правильной последовательности
- Контроль потока - отправитель не должен перегружать получателя
- Контроль перегрузки - отправитель учитывает состояние сети
- Соединение - обмен идет в рамках установленной TCP-сессии

TCP как поток байтов

TCP не передает отдельные сообщения приложения как независимые письма.

Для приложения TCP выглядит как непрерывный поток байтов.

Протокол приложения сам определяет границы команд и сообщений.

Например, HTTP отделяет строки запроса и заголовки своими правилами.

Поэтому TCP подходит для данных, где важны полнота и порядок.

Заголовок TCP: что в нем важно

- Source port - порт отправителя
- Destination port - порт получателя
- Sequence number - номер байта в потоке
- Acknowledgment number - подтверждение полученных данных
- Flags - служебные флаги SYN, ACK, FIN, RST и другие
- Window size - размер окна приема

Заголовок TCP-сегмента

Основные поля TCP и их назначение

Структура TCP-заголовка

- **Порт источника** — 16 бит
- **Порт назначения** — 16 бит
- **Sequence Number** — 32 бита
- **Acknowledgment Number** — 32 бита
- **Data Offset** — 4 бита
- **Reserved** (резерв)
- **Flags** — 9 бит
- **Window Size** — 16 бит
- **Checksum** — 16 бит
- **Urgent Pointer** — 16 бит
- **Options** — переменная длина
- **Data (Payload)** — данные

2. Флаги TCP

- **SYN** — начало соединения
- **ACK** — подтверждение
- **FIN** — завершение
- **RST** — сброс
- **PSH** — срочная передача приложению
- **URG** — срочные данные

Существуют и другие флаги.

3. Что делают основные поля

- **Порты** — какие приложения общаются
- **Sequence Number** — порядок байтов
- **Acknowledgment** — что получено
- **Window Size** — управление потоком
- **Checksum** — проверка ошибок
- **Options** — доп. параметры

4. Как работает TCP



5. Где используется

- HTTP/HTTPS
- SSH
- FTP
- Почта (SMTP, IMAP)
- Веб-приложения и сервисы

Главная идея: TCP использует заголовок для надёжной доставки данных, управления порядком, подтверждениями и контролем потока.

Недостатки TCP

- Перед началом обмена требуется установить соединение.
- Повторная передача увеличивает задержку при потерях.
- Для коротких запросов накладные расходы могут быть заметны.
- TCP не подходит для всех задач реального времени.
- Иногда приложению выгоднее самому контролировать потери и задержки.

3. UDP: User Datagram Protocol

- UDP - User Datagram Protocol, протокол пользовательских датаграмм.
- UDP передает отдельные датаграммы без установки соединения.
- Протокол не гарантирует доставку.
- Протокол не гарантирует порядок получения.
- Протокол не выполняет повторную передачу потерянных данных.

Когда используют UDP

- DNS-запросы, когда нужен быстрый короткий обмен.
- DHCP, когда клиент еще не имеет IP-адреса.
- NTP для синхронизации времени.
- VoIP и видеосвязь, где важна малая задержка.
- Онлайн-игры и телеметрия, где устаревшие данные часто не нужны.

Заголовок UDP

- Source port - порт отправителя.
- Destination port - порт получателя.
- Length - длина UDP-датаграммы.
- Checksum - контрольная сумма для проверки повреждения.
- Заголовок UDP меньше и проще заголовка TCP.

Заголовок UDP-сегмента

Основные поля UDP и их назначение

Размер UDP-заголовка: 8 байт

Порт источника — 16 бит	Порт назначения — 16 бит	Длина — 16 бит	Контрольная сумма — 16 бит
Data (Payload) — данные приложения			

1. Что делают поля

- **Порт источника** — от какого приложения отправлен сегмент
- **Порт назначения** — какому приложению он адресован
- **Длина** — размер заголовка и данных
- **Контрольная сумма** — проверка ошибок

2. Как работает UDP



1. Приложение формирует данные
2. UDP добавляет заголовок
3. Сегмент отправляется без установления соединения
4. Получатель читает порт и передаёт данные приложению

3. Особенности UDP

- без установления соединения
- минимальный заголовок
- без подтверждений и повторной передачи
- быстрее, но менее надёжен, чем TCP

4. Где используется

- DNS
- VoIP
- Видеостриминг
- DHCP
- Онлайн-игры

Главная идея: UDP использует очень короткий заголовок и подходит там, где важны скорость и простота передачи.

Что UDP не делает

- Не устанавливает соединение перед отправкой.
- Не подтверждает получение датаграммы.
- Не восстанавливает потерянные данные.
- Не сортирует датаграммы в правильном порядке.
- Не управляет скоростью отправки так, как это делает TCP.

Почему UDP все равно нужен

- Меньше задержка перед началом обмена.
- Меньше служебных данных в заголовке.
- Подходит для широковещательных и некоторых служебных протоколов.
- Позволяет приложению самому выбирать логику надежности.
- Удобен для задач, где скорость важнее гарантированной доставки.

TCP и UDP: профессиональное сравнение

- TCP выбирают, когда потеря данных недопустима.
- UDP выбирают, когда важны скорость, простота и низкая задержка.
- TCP удобен для файлов, сайтов, удаленного доступа и почты.
- UDP удобен для DNS, DHCP, времени, голоса и видео.
- Выбор протокола определяется требованиями приложения.

4. TCP handshake

TCP handshake - процедура установления TCP-соединения.

Handshake переводится как рукопожатие.

На практике это обмен служебными сегментами перед передачей данных.

Классический TCP handshake состоит из трех шагов.

Его часто называют three-way handshake.

Шаг 1: SYN

- Клиент отправляет серверу сегмент с флагом SYN.
- SYN означает synchronize, синхронизация.
- Клиент предлагает начать соединение.
- Клиент сообщает начальный sequence number.
- Сервер должен получить этот сегмент на слушающем порту.

Шаг 2: SYN-ACK

- Сервер отвечает сегментом SYN-ACK.
- ACK означает acknowledgment, подтверждение.
- Сервер подтверждает получение SYN клиента
- Сервер сообщает свой начальный sequence number.
- Этот ответ показывает, что порт доступен и служба отвечает.

Шаг 3: ACK

- Клиент отправляет ACK в ответ на SYN-ACK.
- После этого соединение считается установленным.
- Приложения могут передавать полезные данные.
- Для администратора это признак успешной доступности TCP-порта.
- Если handshake не завершился, нужно искать причину на сети, firewall или службе.

TCP three-way handshake

Как клиент и сервер устанавливают соединение перед передачей данных

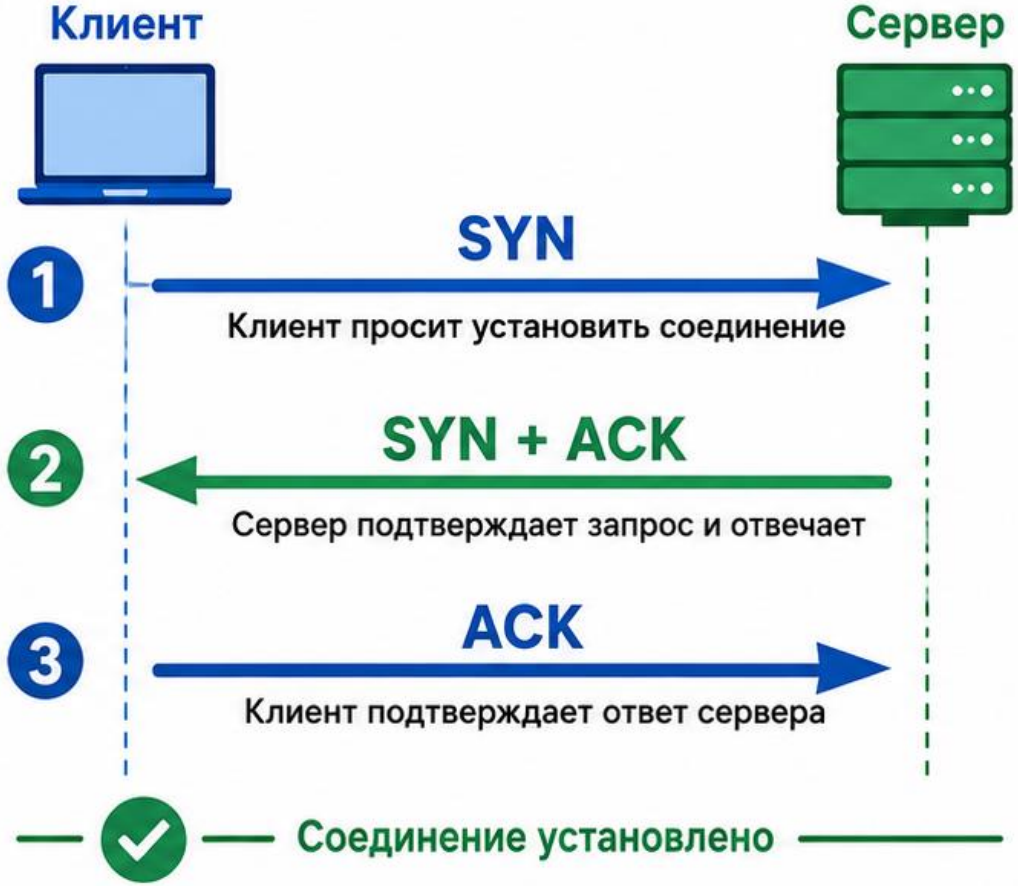
1 Что это такое



TCP сначала устанавливает соединение, чтобы стороны согласовали параметры обмена.

3 Какие флаги участвуют

SYN	ACK	SYN+ACK
начало соединения	подтверждение	ответ сервера



2 Что происходит после этого

- можно передавать данные
- обмен идёт надёжно
- используются номера последовательности и подтверждения

4 Где используется

- HTTP/HTTPS
- SSH
- FTP
- почта
- веб-приложения

Главная идея

TCP three-way handshake — это трёхшаговое установление соединения: SYN → SYN-ACK → ACK.

Что может нарушить TCP handshake

- На сервере не запущена нужная служба.
- Служба слушает другой порт.
- Windows Firewall или nftables блокирует соединение.
- Маршрутизатор или ACL запрещает трафик.
- NAT перенаправляет порт не на тот внутренний сервер.
- Сервер перегружен или недоступен по сети.

5. Метод скользящего окна на уровне назначения

Скользящее окно - механизм управления объемом неподтвержденных данных TCP;

Окно показывает, сколько данных можно отправить без ожидания нового ACK;

ACK сообщает, какие данные получатель уже принял;

Если окно маленькое, передача замедляется;

Если окно большое, TCP может эффективнее использовать канал;

Зачем нужно окно TCP

- Получатель может быть медленнее отправителя;
- Буфер получателя может заполниться;
- Сеть может терять сегменты при перегрузке;
- TCP должен передавать данные быстро, но без разрушения соединения;
- Окно помогает согласовать скорость отправителя и получателя;

Sequence number и ACK

Sequence number нумерует байты в TCP-потоке.

ACK подтверждает получение данных до определенного номера.

Если сегмент потерян, подтверждение не продвигается вперед.

Отправитель понимает, что часть данных нужно отправить снова.

Так TCP восстанавливает поток после потерь.

ПРИНЦИП СКОЛЬЗЯЩЕГО ОКНА TCP

TCP обеспечивает надежную передачу данных, используя механизм скользящего окна.

ОТПРАВИТЕЛЬ
Отправляет данные в пределах окна

СКОЛЬЗЯЩЕЕ ОКНО
Определяет диапазон номеров сегментов, которые могут быть отправлены без получения подтверждения (ACK)

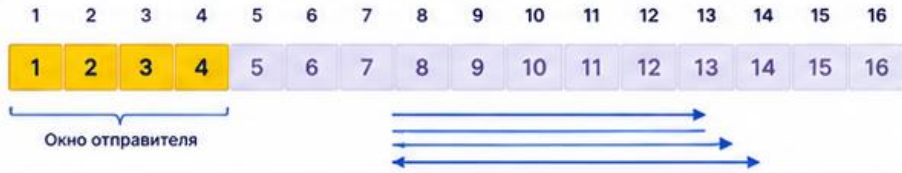
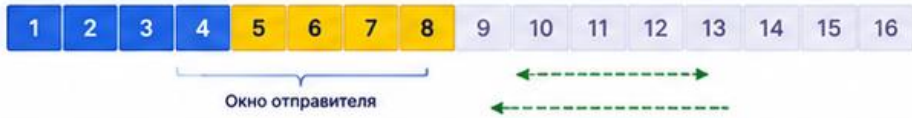
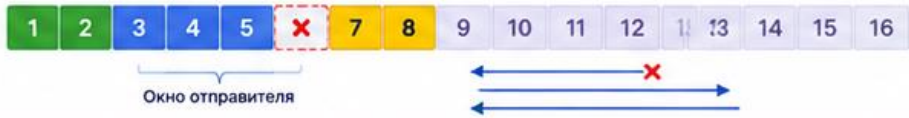
ПОЛУЧАТЕЛЬ
Подтверждает получение данных (ACK)

ЛЕГЕНДА СОСТОЯНИЙ ОКНА

- Отправлено и подтверждено (ACK получен)
- Отправлено, ожидает подтверждения
- Можно отправлять (внутри окна)
- Нельзя отправлять (вне окна)

Обозначения

- Передача данных (сегмент)
- Подтверждение (ACK)
- Потеря сегмента
- Таймер (на каждый сегмент)

ОТПРАВИТЕЛЬ (КЛИЕНТ)	ПОСЛЕДОВАТЕЛЬНОСТЬ НОМЕРОВ (SEQ)	ПОЛУЧАТЕЛЬ (СЕРВЕР)
1 Начальное состояние Размер окна = 4 (может быть отправлено до 4 сегментов)		Получено
2 Отправлены 4 сегмента Сегменты 1-4 отправлены, ожидаем ACK		Получено 1 2 3 4
3 Получены ACK для 1-2 Окно сдвигается вправо на 2 позиции		Получено 1 2
4 Потеря сегмента Сегмент 6 потерян (нет ACK)		Получено 1 2 3 4 5 Получатель ожидает сегмент 6, поэтому не подтверждает последующие сегменты
5 Таймаут и повторная передача Таймер сегмента 6 истек, сегмент 6 отправлен повторно		Получено 1 2 3 4 5 6
6 Окно сдвигается дальше Получены ACK для 6, окно сдвигается и можно отправлять новые данные		Получено 1 2 3 4 5 6

Окно «скользит» вперед по мере получения подтверждений, обеспечивая эффективное и надежное использование канала связи.

КАК РАБОТАЕТ СКОЛЬЗЯЩЕЕ ОКНО?

1. Отправитель может отправлять несколько сегментов подряд, не дожидаясь подтверждения для каждого (в пределах окна).
2. Получатель отправляет ACK с номером следующего ожидаемого сегмента (кумулятивное подтверждение).
3. При получении ACK окно сдвигается вправо, освобождая место для новых сегментов.
4. Если сегмент потерян (не получен ACK), срабатывает таймер и сегмент отправляется повторно.
5. Размер окна (rwnd) может динамически изменяться и зависит от буфера получателя.
6. Это повышает пропускную способность сети и надежность передачи.

ВАЖНЫЕ ПОНЯТИЯ

SEQ — Sequence Number
Порядковый номер первого байта в сегменте.

ACK — Acknowledgment Number
Номер следующего ожидаемого байта.

rwnd — Receive Window
Размер окна, объявляемый получателем.

Таймер
Запускается для каждого отправленного сегмента.

Кумулятивное подтверждение
ACK подтверждает все байты до указанного номера (не включая его).

Что видит администратор

- При потерях пакетов TCP может работать медленно;
- При высокой задержке соединение может открываться долго;
- При малом окне передача больших файлов замедляется;
- Ошибка может выглядеть как медленный сайт или медленная общая папка;
- Для анализа используют ping, traceroute, ss, netstat, Wireshark или tcpdump;

6. Socket: сочетание IP-адреса, протокола и порта

Socket - программная конечная точка сетевого обмена;

Сокет связывает приложение с IP-адресом, протоколом и портом;

Для сервера сокет обычно означает адрес и порт, которые служба слушает;

Для клиента сокет включает временный исходящий порт;

В TCP-соединении важна пара клиентского и серверного сокетов;

Порт как адрес приложения

- Порт - числовой идентификатор сетевого приложения на узле;
- Порт находится на транспортном уровне, а не на сетевом;
- Один сервер может иметь один IP-адрес и много служб на разных портах;
- Например, 22 для SSH, 80 для HTTP, 443 для HTTPS;
- Номер порта сам по себе не гарантирует, что там работает ожидаемая служба;

Серверный и клиентский сокет

Сервер слушает известный порт, например TCP 443;

Клиент открывает временный порт из динамического диапазона;

Пример: клиент 192.168.1.50:51544 подключается к серверу 192.168.1.10:443;

Для TCP соединение определяется парой адресов и портов;

Поэтому к одному серверному порту могут подключаться много клиентов;

TCP-СОЕДИНЕНИЕ ЧЕРЕЗ СОКЕТЫ

Схема взаимодействия клиентского и серверного приложений через сокеты TCP

КЛИЕНТ (активная сторона)

1 socket()
Создание сокета
`sock = socket(AF_INET, SOCK_STREAM)`

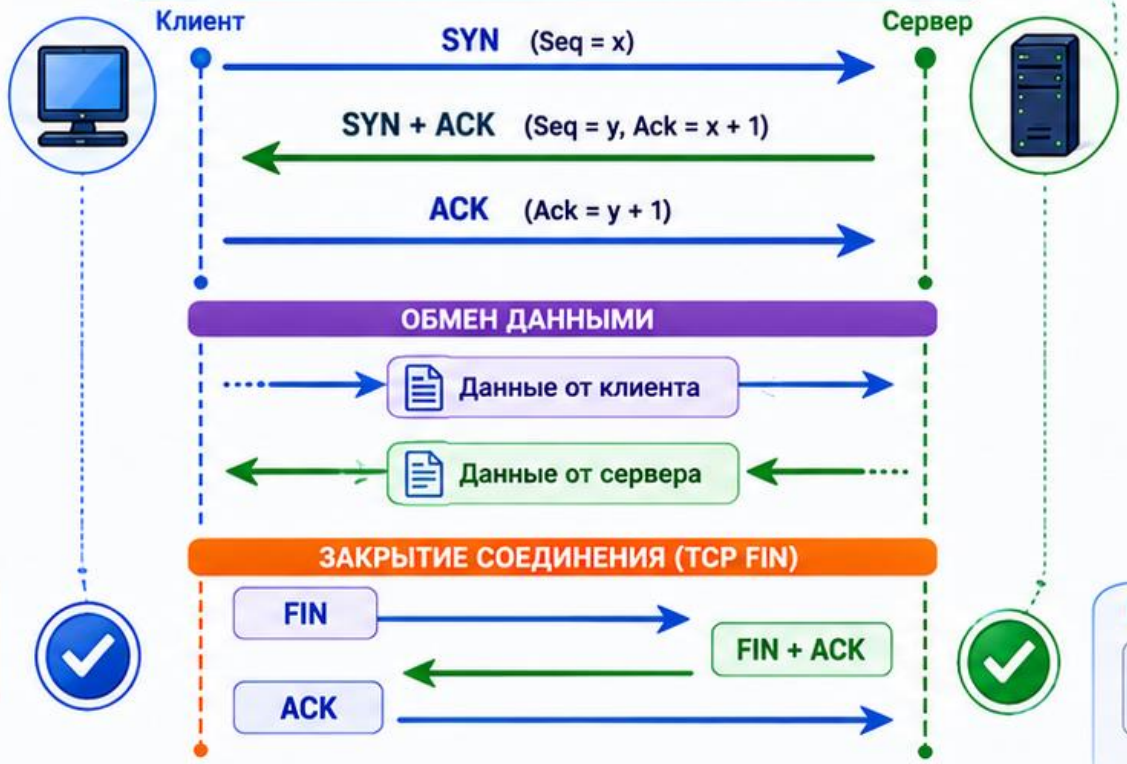
2 connect()
Соединение с сервером
`connect(sock, 10.0.0.5:80)`

3 send()/recv()
Передача данных
`send(sock, ...)`
`recv(sock, ...)`

4 close()
Закрытие сокета
`close(sock)`



УСТАНОВЛЕНИЕ TCP-СОЕДИНЕНИЯ (Three-Way Handshake)



ОБМЕН ДАННЫМИ

ЗАКРЫТИЕ СОЕДИНЕНИЯ (TCP FIN)

СЕРВЕР (пассивная сторона)

1 socket()
Создание сокета
`listen_sock = socket(AF_INET, SOCK_STREAM)`

2 bind()
Привязка к адресу и порту
`bind(listen_sock, 0.0.0.0:80)`

3 listen()
Переход в режим ожидания
`listen(listen_sock, backlog)`

4 accept()
Принятие соединения
`conn_sock = accept(listen_sock)`

5 send()/recv()
Обмен данными
`send(conn_sock, ...)`
`recv(conn_sock, ...)`

6 close()
Закрытие соединения
`close(conn_sock)`

ЧТО ТАКОЕ СОКЕТ?

Сокет — это конечная точка взаимодействия приложений по сети через транспортный протокол.

СОСТОЯНИЯ TCP-СОЕДИНЕНИЯ



АДРЕС СОКЕТА

Клиентский сокет	Серверный сокет
IP: 192.168.1.10	IP: 10.0.0.5
Порт: 54321	Порт: 80

ОСНОВНЫЕ ФУНКЦИИ СОКЕТОВ (BSD-API)

socket()	bind()	listen()	accept()	connect()	send()/recv()	close()
Создание	Привязка	Ожидание	Принятие	Соединение	Обмен данными	Закрытие

ВАЖНО

- Каждая связь уникальна по паре: IP + порт отправителя ↔ IP + порт получателя
- TCP обеспечивает надежную, упорядоченную и потоковую передачу данных
- Сокеты используются большинством сетевых приложений (HTTP, FTP, SMTP и др.)

SOCKET: СОЧЕТАНИЕ IP-АДРЕСА, ПРОТОКОЛА И ПОРТА

Socket (сокет) — это **конечная точка соединения** в сети. Он однозначно идентифицирует приложение на устройстве.

ЧТО ТАКОЕ SOCKET?

Socket — это комбинация из трёх элементов:

IP-адрес
Уникальный адрес устройства в сети (IPv4 или IPv6)

+

Протокол
Протокол транспортного уровня: TCP или UDP

+

Порт
Число от 0 до 65535, которое определяет конкретное приложение или службу на устройстве

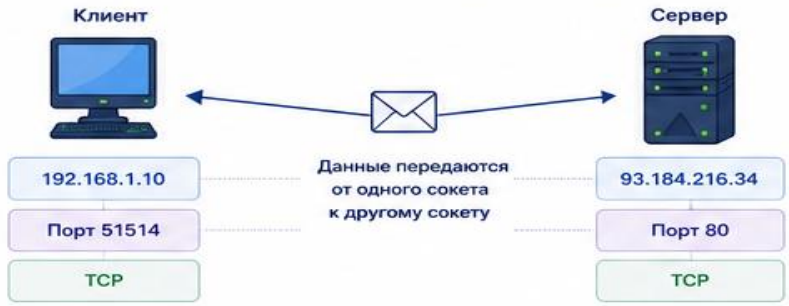
Формально запись socket:

<IP-адрес, протокол, порт>

Пример: 192.168.1.10:80/TCP

КАК РАБОТАЕТ SOCKET?

Чтобы данные дошли до нужного приложения, недостаточно знать только IP-адрес.
На одном устройстве может работать много приложений одновременно.



Идентификация соединения:

(192.168.1.10, TCP, 51514) ⇄ (93.184.216.34, TCP, 80)

Так операционная система понимает, какому приложению передать данные.

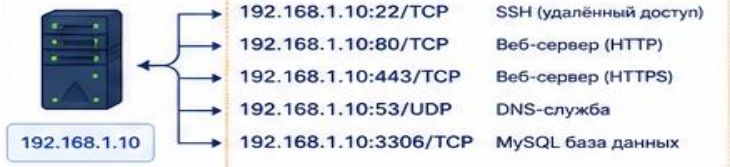
ПРИМЕРЫ СОКЕТОВ И СЛУЖБ

Служба (приложение)	Протокол	Порт	Пример socket
HTTP (веб)	TCP	80	93.184.216.34:80/TCP
HTTPS (веб по SSL)	TCP	443	93.184.216.34:443/TCP
SMTP (почта отправка)	TCP	25	mail.example.com:25/TCP
POP3 (почта получение)	TCP	110	mail.example.com:110/TCP
DNS (запросы)	UDP	53	8.8.8.8:53/UDP
SSH (удалённый доступ)	TCP	22	192.168.1.1:22/TCP
Игровой сервер	UDP	27015	203.0.113.5:27015/UDP

Порт + протокол указывают на службу.
IP-адрес указывает, на каком устройстве она запущена.

ОДИН IP — МНОГО СОКЕТОВ

На одном устройстве может быть множество сокетов — разные приложения используют разные порты и протоколы.



Один IP может обслуживать тысячи соединений благодаря различным портам и протоколам.

СОЕДИНЕНИЕ МЕЖДУ СОКЕТАМИ (TCP)

Каждое TCP-соединение уникально и определяется парой сокетов:

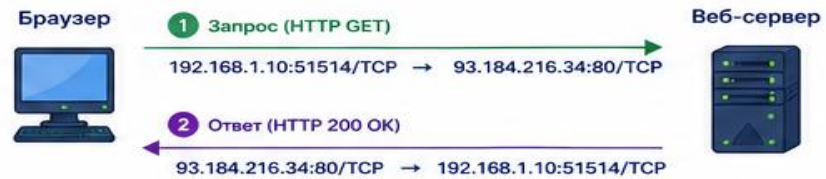


Пока соединение активно, данные обмениваются между этими сокетами.

Ключевое правило
Ответ всегда отправляется обратно на тот же локальный порт клиента.
Сервер различает клиентов по разным комбинациям:
(IP клиента, порт клиента, протокол).

ПРИМЕР ОБМЕНА ДАННЫМИ (HTTP)

Браузер открывает сайт example.com (IP: 93.184.216.34)



Сервер отвечает на тот же порт клиента (51514), на который пришёл запрос.

TCP vs UDP В СОКЕТАХ

TCP
- ✓ Устанавливает соединение (рукопожатие)
- ✓ Надёжная, гарантирует доставку и породок
- ✓ Используется для веб, почты, файлов и др.

Пример сокета: 192.168.1.10:443/TCP

UDP
- ✓ Соединение не устанавливается
- ✓ Доставка без гарантии, быстрее
- ✓ Используется для DNS, VoIP, стриминга, игр

Пример сокета: 8.8.8.8:53/UDP

ДИАПАЗОН ПОРТОВ

- **Известные (Well-Known): 0 – 1023**
  Системные службы (HTTP – 80, SSH – 22 и др.)
- **Зарегистрированные (Registered): 1024 – 49151**
  Приложения сторонних разработчиков
- **Динамические (Ephemeral): 49152 – 65535**
  Кратковременные порты для клиентов (обычно)

Пример динамического порта
51514

ОС выбирает свободный порт из этого диапазона для нового соединения.

ИТОГ

- ✓ Socket = IP-адрес + протокол + порт
- ✓ Он определяет, какому приложению доставить данные
- ✓ Без порта невозможно различить службы на одном устройстве
- ✓ TCP соединяет сокеты и обеспечивает надёжную передачу данных

<IP, протокол, порт>
↓
УНИКАЛЬНЫЙ СОКЕТ
= адрес приложения в сети

Состояния TCP-соединений

- LISTEN - служба ожидает входящие подключения;
- ESTABLISHED - соединение установлено и используется;
- TIME_WAIT - соединение закрыто, но запись временно сохраняется;
- SYN_SENT - клиент отправил SYN и ждет ответа;
- CLOSE_WAIT - удаленная сторона закрыла соединение, локальная еще не завершила закрытие;

Как посмотреть сокеты в ОС

В Linux используют `ss -tulpn`
Ключ `-t` показывает TCP-сокеты
Ключ `-u` показывает UDP-сокеты
Ключ `-l` показывает слушающие порты
В Windows используют `netstat -ano` или `Get-NetTCPConnection`

Как посмотреть сокеты в ОС

```
[noma@mamayaprogrammist ~]$ ss -tulpn
Netid State   Recv-Q  Send-Q          Local Address:Port       Peer Address:Port
udp   UNCONN  0       0                     0.0.0.0:51042           0.0.0.0:*
udp   UNCONN  0       0               192.168.122.1:53              0.0.0.0:*
udp   UNCONN  0       0             0.0.0.0%virbr0:67               0.0.0.0:*
udp   UNCONN  0       0           10.0.2.15%enp0s3:68               0.0.0.0:*
udp   UNCONN  0       0                     0.0.0.0:111             0.0.0.0:*
udp   UNCONN  0       0                     0.0.0.0:5353            0.0.0.0:*
udp   UNCONN  0       0                        [::]:59413              [::]:*
udp   UNCONN  0       0                        [::]:111                [::]:*
udp   UNCONN  0       0                        [::]:5353               [::]:*
tcp   LISTEN  0       128                   0.0.0.0:111             0.0.0.0:*
tcp   LISTEN  0       32              192.168.122.1:53              0.0.0.0:*
tcp   LISTEN  0       128                   0.0.0.0:22              0.0.0.0:*
tcp   LISTEN  0       5                   127.0.0.1:631             0.0.0.0:*
tcp   LISTEN  0       128                      [::]:111                [::]:*
tcp   LISTEN  0       128                      [::]:22                 [::]:*
tcp   LISTEN  0       5                       [::1]:631                [::]:*
[noma@mamayaprogrammist ~]$ _
```

Состояния UDP-портов в Linux

UNCONN (Unconnected) — Ожидание пакетов
Что означает: Порт успешно открыт программой, привязан к системе и находится в режиме ожидания входящих пакетов.
это стандартное рабочее состояние для большинства UDP-серверов (DNS, DHCP, NTP).

ESTAB (Established) — Зафиксированный адрес
Что означает: Программа внутри своего кода вызвала функцию connect() для UDP-сокета.
Это состояние означает лишь то, что приложение заблокировало этот сокет строго под одного конкретного получателя.

7. Well-known ports популярных сервисов

Well-known ports - общеизвестные порты популярных служб.

Диапазон well-known ports: 0-1023.

Эти номера закреплены за стандартными сервисами.

Администратор должен знать часто встречающиеся порты.

Знание портов ускоряет настройку firewall и диагностику.

Порты удаленного доступа и файлов

TCP 22 - SSH, защищенное удаленное администрирование Linux.
TCP 3389 - RDP, удаленный рабочий стол Windows.
TCP 445 - SMB, общие папки и доменные службы Windows.
TCP 21 - FTP, управление передачей файлов.
TCP 20 - FTP data, передача данных в классическом FTP.

Порты веб-служб и почты

TCP 80 - HTTP, веб без шифрования.
TCP 443 - HTTPS, веб с TLS-шифрованием.
TCP 25 - SMTP, передача почты между серверами.
TCP 110 - POP3, получение почты клиентом.
TCP 143 - IMAP, работа с почтовым ящиком на сервере.
TCP 993 - IMAPS, IMAP поверх TLS.

Порты инфраструктурных служб

TCP/UDP 53 - DNS, разрешение имен.
UDP 67 и UDP 68 - DHCP, автоматическая выдача IP-параметров.
UDP 123 - NTP, синхронизация времени.
TCP/UDP 389 - LDAP, каталог пользователей и объектов.
TCP 636 - LDAPS, LDAP поверх TLS.

Порты не заменяют понимание сервиса

Один и тот же сервис может быть перенесен на нестандартный порт.

На стандартном порту может работать другой сервис.

Firewall может разрешать порт только из определенной сети.

Служба может слушать только localhost, а не сетевой интерфейс.

Проверка порта должна дополняться проверкой ответа приложения.

Безопасность открытых портов

- Открытый порт означает потенциальную точку входа;
- Ненужные службы должны быть остановлены или закрыты firewall;
- Доступ к административным портам нужно ограничивать;
- Публичные сервисы требуют обновлений и журналирования;
- Сканирование портов помогает увидеть реальную поверхность атаки;

8. Отличие проверки IP-связности от проверки доступности приложения

IP-связность показывает, что узлы могут обмениваться пакетами на сетевом уровне.

Приложение может быть недоступно даже при рабочей IP-связности.

Ping использует ICMP, а не TCP или UDP.

ICMP - Internet Control Message Protocol, протокол служебных сетевых сообщений.

Успешный ping не доказывает, что TCP 443 или UDP 53 доступны.

Ситуация: ping работает, сайт не открывается

- Маршрут до сервера есть;
- Сервер отвечает на ICMP Echo Request;
- Веб-сервер может быть остановлен;
- TCP-порт 80 или 443 может быть закрыт firewall;
- DNS-имя может указывать не на тот IP-адрес;

Ситуация: ping не работает, сайт открывается

- ICMP может быть запрещен правилами безопасности;
- Некоторые серверы специально не отвечают на ping;
- TCP 443 может быть разрешен, а ICMP запрещен;
- Поэтому нельзя делать вывод только по ping;
- Нужно проверять именно нужный протокол и порт;

Уровни проверки доступности

- Проверка адресации: правильный IP-адрес и маска;
- Проверка маршрута: есть ли путь до узла;
- Проверка порта: принимает ли служба TCP или UDP;
- Проверка протокола приложения: отвечает ли HTTP, DNS, SSH или другой сервис;
- Проверка доступа: хватает ли прав, сертификатов и учетных данных;

ДИАГРАММА: УРОВНИ ПРОВЕРКИ СЕТЕВОГО СЕРВИСА

Проверка выполняется по уровням модели TCP/IP: от нижнего (физического) до прикладного.

УРОВНИ ПРОВЕРКИ

7 ПРИКЛАДНОЙ
HTTP, HTTPS, DNS, SMTP, FTP, SSH и др.
Проверка работы конкретного сетевого сервиса и корректности данных для пользователя.

6 ПРЕДСТАВИТЕЛЬНЫЙ
SSL/TLS, кодеки, сжатие
Проверка шифрования, кодирования, сжатия и преобразования данных.

5 СЕАНСОВЫЙ
NetBIOS, RPC, SIP
Проверка установки, поддержания и завершения сеансов связи между приложениями.

4 ТРАНСПОРТНЫЙ
TCP, UDP
Проверка соединений TCP/UDP, портов, управления потоком, повторной передачи.

3 СЕТЕВОЙ
IP (IPv4/IPv6), ICMP, маршрутизация
Проверка IP-адресации, маршрутизации, доступности узлов, ICMP-сообщений.

2 КАНАЛЬНЫЙ
Ethernet, Wi-Fi, VLAN, PPP
Проверка локальной связи, MAC-адресов, VLAN, обнаружения ошибок на канальном уровне.

1 ФИЗИЧЕСКИЙ
Кабели, разъёмы, электрические и оптические сигналы
Проверка целостности кабелей, соединений, уровня сигнала, индикации на устройствах.

КАК ПРОВЕРЯЕМ? (ПРИМЕРЫ ИНСТРУМЕНТОВ)

Браузер, curl, Postman, nslookup/dig, почтовый клиент, FTP-клиент, SSH-клиент	
openssl, браузер (SSL-информация), проверка сертификатов, расшифровка трафика	
telnet (устар.), RPC-тесты, SIP-телефон, тесты NetBIOS	
telnet / nc, Test-NetConnection, проверка портов, Wireshark	
ping, tracert/traceroute, mtr, проверка маршрутов (route), ICMP (эхо, недоступность)	ping, ICMP
arp -a, mac-таблица, Wireshark, проверка VLAN, STP	ARP
Проверка кабеля (тестер), индикаторы линка, измерение сигнала (OTDR/мультиметр)	

ПОСЛЕДОВАТЕЛЬНОСТЬ ПРОВЕРКИ

1. **Физический уровень**
   Есть ли физическое соединение?
   Горят ли индикаторы?
   Цел ли кабель?
2. **Канальный уровень**
   Получен ли MAC-адрес?
   Есть ли связь в локальной сети?
   Нет ли ошибок?
3. **Сетевой уровень**
   Доступен ли IP-адрес?
   Пингуется ли шлюз/узел?
   Есть ли маршрут?
4. **Транспортный уровень**
   Открыт ли порт?
   Устанавливается ли TCP/UDP соединение?
5. **Сеансовый уровень**
   Устанавливается ли сеанс?
   Поддерживается ли он?
6. **Представительный уровень**
   Корректно ли шифруются и декодируются данные?
7. **Прикладной уровень**
   Работает ли сервис так, как ожидает пользователь?

ПРИМЕР: ПРОВЕРКА ВЕБ-СЕРВИСА (HTTP/HTTPS)

7. **Браузер открывает сайт** — Страница загружается, данные отображаются. ✓
6. **Проверка SSL-сертификата** — Соединение защищено, сертификат действителен. ✓
5. **Сеанс HTTP установлен** — Соединение поддерживается на время работы сайта. ✓
4. **TCP соединение установлено** — Порт 80/443 открыт, данные передаются. ✓
3. **IP-адрес сайта доступен** — Пинг проходит, маршрут найден. ✓
2. **Есть локальная связь** — MAC-адрес получен, ошибок нет. ✓
1. **Кабель подключён** — Линк активен, сигнал в норме. ✓

ПРИНЦИП

Если на каком-либо уровне проверка не проходит – вышележащие уровни работать не будут.

СОВЕТЫ

- Проверяйте последовательно, не пропуская уровни.
- Используйте подходящий инструмент для каждого уровня.
- Фиксируйте результаты и переходите к следующему уровню только при успехе.

РЕЗУЛЬТАТ

Поэтапная проверка позволяет быстро найти причину проблемы и восстановить работу сервиса.

9. Диагностика транспортного уровня

- Диагностика начинается с определения нужного сервиса и порта;
- Нужно понять, используется TCP или UDP;
- Затем проверяют, слушает ли служба порт на сервере;
- После этого проверяют доступность порта с клиента;
- Результаты нужно фиксировать: команда, адрес, порт, ответ, вывод;

Базовый алгоритм диагностики

- Уточнить имя сервиса и ожидаемый порт;
- Проверить IP-адрес сервера и маршрут до него;
- Проверить, запущена ли служба на сервере;
- Проверить, слушает ли служба нужный интерфейс и порт;
- Проверить firewall на сервере и промежуточных устройствах;
- Проверить ответ приложения, а не только открытие порта;

Команды Linux для TCP и UDP

- **ss -tulpen** показывает слушающие TCP и UDP-порты;
- **curl http://server** проверяет HTTP-ответ;
- **nc -vz server 22** проверяет TCP-подключение к порту;
- **dig @server name** проверяет DNS-запрос;
- **tcpdump** позволяет увидеть реальные пакеты на интерфейсе;

Команды Windows для TCP

- **Test-NetConnection server -Port 443** проверяет TCP-порт;
- **netstat -ano** показывает соединения и PID процессов;
- **Get-NetTCPConnection** показывает TCP-соединения PowerShell;
- **Resolve-DnsName** name проверяет разрешение DNS-имени;
- Журнал событий помогает найти ошибки службы или firewall;

Как читать типовые результаты

- **Connection refused** - узел доступен, но порт закрыт или служба не слушает;
- **Timeout** - нет ответа, часто причина в firewall, маршруте или фильтрации;
- **Name not resolved** - не работает разрешение имени
- **TLS error** - порт доступен, но есть проблема сертификата или шифрования;
- **Authentication failed** - сервис отвечает, но доступ запрещен учетными данными;

ПУТЬ ДИАГНОСТИКИ ТРАНСПОРТНОГО УРОВНЯ

Пошаговый подход для поиска и устранения проблем на транспортном уровне (TCP/UDP).

1 ПРОВЕРЬ СВЯЗЬ (IP-УРОВЕНЬ)

Есть ли доступность до удалённого хоста?



```
ping <IP-адрес>
traceroute/tracert <IP>
```

Ожидаемый результат:
Ответы от хоста или маршрут до него

2 ПРОВЕРЬ ПОРТ

Открыт ли порт на удалённом хосте?



```
nc -vz <IP> <порт>
или
telnet <IP> <порт>
```

Ожидаемый результат:
Соединение установлено или порт открыт

3 ПРОВЕРЬ СЛУЖБУ НА УДАЛЁННОМ ХОСТЕ

Работает ли нужная служба на целевом хосте?



Проверка службы:
`systemctl status <service>`
или
`netstat -an | findstr :<порт>`

Ожидаемый результат:
Служба запущена и слушает на нужном порту

4 ПРОВЕРЬ СОЕДИНЕНИЕ (TCP)

Устанавливается ли TCP-соединение?



Проверка состояния:
`netstat -an | find "ESTABLISHED"`
или
`ss -tan | grep <порт>`

Ожидаемый результат:
Состояние ESTABLISHED (для TCP)

5 ПРОВЕРЬ ПЕРЕДАЧУ ДАННЫХ

Передаются ли данные без ошибок и потерь?



Тест данных:
`iperf -c <IP> -p <порт>`
или
передача файла/запрос

Ожидаемый результат:
Данные передаются, скорость в норме

6 АНАЛИЗИРУЙ И УСТРАНЯЙ

Есть ли ошибки, задержки или потери?



Анализ:
Wireshark, tcpdump, ss, netstat, журналы, счётчики ошибок

Ожидаемый результат:
Проблема найдена и устранена

ЕСЛИ ПРОБЛЕМА ОБНАРУЖЕНА

Нет ответа на ping

- Проверить кабель и Link
- Проверить маршрутизацию
- Проверить ACL/Брандмауэр
- Проверить настройки IP

Цель: восстановить IP-связность

Порт недоступен

- Проверить брандмауэр/ACL
- Проверить правильность порта
- Проверить слушающую службу

Цель: открыть порт или запустить службу

Соединение не устанавливается (SYN без ответа)

- Проверить фильтры (FW/ACL)
- Проверить перегрузку сети
- Проверить состояние сервера

Цель: обеспечить возможность установления соединения

Соединение сбрасывается (RST)

- Служба не принимает соединения
- Неправильные параметры клиента
- Политики безопасности

Цель: устранить причину сброса соединения

Соединение установлено, но данные не передаются

- Проверить приложение/протокол
- Проверить MTU, фрагментацию
- Проверить потери и задержки

Цель: восстановить стабильную передачу данных

ПОЛЕЗНЫЕ СОСТОЯНИЯ TCP (netstat/ss)

LISTEN	SYN-SENT	SYN-RECEIVED	ESTABLISHED	FIN-WAIT	TIME-WAIT	CLOSED
Ожидание входящих соединений	Отправлен SYN, ожидание SYN-ACK	Получен SYN, ожидание ACK	Соединение установлено	Ожидание закрытия соединения	Ожидание завершения старых пакетов	Соединение закрыто

КЛЮЧЕВЫЕ ПРИНЦИПЫ

- Диагностика идёт от нижнего уровня к верхнему.
- Проверяйте по одному параметру за раз.
- Сравнивайте с рабочим хостом/сервисом.
- Фиксируйте результаты на каждом этапе.



СОВЕТ



Сначала убедитесь, что сеть работает, затем проверяйте транспортный уровень, а потом — приложение.

ИНСТРУМЕНТЫ И КОМАНДЫ

ping
Проверка доступности хоста (ICMP echo).

traceroute / tracert
Показывает маршрут до хоста.

nc (netcat)
Проверка TCP/UDP портов, установка соединения.

telnet
Проверка TCP соединений (устаревший, но полезный).

netstat / ss
Показ активных соединений, прослушиваемых портов и состояний.

iperf
Измерение пропускной способности и качества канала.

Wireshark / tcpdump
Анализ пакетов и диагностика проблем на транспортном уровне.

ЧАСТЫЕ ПРОБЛЕМЫ ТРАНСПОРТНОГО УРОВНЯ

- **Порт закрыт / фильтруется** — Брандмауэр, ACL, неправильная служба
- **Служба не запущена** — Приложение не слушает порт
- **Сброс соединения (RST)** — Приложение отклонило соединение
- **Таймаут соединения** — Потеря пакетов, маршрутизация, фильтры
- **Медленная передача данных** — Перегрузка канала, высокая задержка, ограничения TCP окна

Типовые ошибки начинающих администраторов

- Проверять только ping и считать сервис исправным;
- Открывать порт в firewall, но не запускать службу;
- Запускать службу только на 127.0.0.1 вместо сетевого интерфейса;
- Путать TCP и UDP для одного и того же номера порта;
- Не фиксировать команды и результаты проверки;

Итоги лекции

- Транспортный уровень связывает приложения на разных узлах;
- TCP обеспечивает надежный упорядоченный обмен;
- UDP обеспечивает быстрый обмен без гарантий доставки;
- Порт указывает, к какой службе должны попасть данные;
- Socket связывает адрес, протокол и порт;
- Диагностика сервиса требует проверки не только IP, но и порта, службы и приложения;

Контрольные вопросы

1. Что делает транспортный уровень?
2. Чем порт отличается от IP-адреса?
3. Что означает TCP и какую задачу выполняет этот протокол?
4. Что означает UDP и почему он не гарантирует доставку?
5. Что происходит во время TCP handshake?
6. Для чего нужен метод скользящего окна?
7. Что такое socket?
8. Почему ping может работать, а сайт не открываться?
9. Какие порты популярных служб должен знать администратор?
10. Какие команды можно использовать для проверки TCP-порта?